

開発レポート Development Report

カメラ認識技術検証アプリ「Vision Studio」の開発

— 色検知・追跡・ライン・アーム接近・コード読取の統合 GUI —

Vision Studio: An Integrated GUI for Camera-Based Color Detection, Tracking, Line Following, Arm Approach and Code Reading

レポート番号	MTIC-DR-2026-007
団体	NPO 法人松戸技術革新センター M-TIC 松戸ロボット研究会
著者	大城 公平
発行日	2026 年 9 月 26 日
対象	Vision Studio v0.2.3 (ソフトウェア)
分類	開発レポート／画像認識ソフト
機体状態の基準日	2026 年 9 月 21 日

本レポートの数値と機体状態について 本検証系は研究開発段階にあり随時変更されるため、記載の仕様は基準日時点のものであり、現在の実機と一致することを保証しない。数値には種別を付す：**公称**＝カタログ・設計値、**実測**＝実際に計測した値（記載桁が測定分解能）、**概算**＝目測・推定値。

概要

ロボットにカメラ認識を導入するには、認識の状態を人が確認しながら調整し、その結果をログや制御系へ渡せる道具が必要である。「Vision Studio」（正式名称：松戸技術革新センター カメラ認識技術検証アプリケーション）は、色検知・物体追跡・ライントレース・アーム接近・QR／バーコード読取の5モードを1つのGUIで切り替え、認識結果をJSONLログ・UDP・シリアルで出力するアプリである。プログラムが得意でない人でもGUIで操作できることと、Raspberry Pi 4Bで動く軽さを優先し、pillow等の追加ライブラリに依存しない構成とした。アーム接近モードは、水平多関節（SCARA型）アームの先端付近のカメラで対象物を捉え、画面内の位置と大きさからアームを対象へ近づけるための情報を出力する。ライントレース専用のソフトは別レポート（MTIC-DR-2026-009）で報告する。各モードの認識精度と処理速度はまだ計測しておらず、次段階で搭載先のCWM・CBV上で計測してデータをもとに評価したい。

1. 背景と目的

1.1 開発背景

M-TICが研究開発しているピッキングロボットCWMと運搬ロボットCBVは、Raspberry Pi 4BとPico 3台（CAN接続）で構成され、カメラを搭載している。物体検知・ライントレース・物体追跡・アームの物体接近をカメラで行えるかを検証するにあたり、認識状態を目で見て調整でき、結果を制御系に渡せるGUIツールが必要になった。

1.2 目的と目標

ロボット本体を動かすことよりも、カメラ認識の状態をログや制御系に反映できる出力まで落とし込むことを目的とする。GUIで操作でき、Raspberry Pi 4Bで動く軽さを条件とする。

目標：5つの認識モードを1つのGUIで切り替えられ、認識結果がJSONL・UDP・シリアルのいずれでも出力されること。

2. システム構成

2.1 開発環境と基本的な作り方

本アプリは Python で記述し、開発環境には Thonny を用いた。Thonny は初学者向けに設計された統合開発環境で、Windows 上で作成したプログラムを多くの場合そのまま Raspberry Pi 4B でも実行できる（使うライブラリによっては OS ごとの違いに注意が必要）。カメラ映像の取得と画像処理には標準的な画像処理ライブラリ OpenCV を用い、画面（GUI）は Python 標準の Tkinter で構成した。処理結果は UDP（同じネットワーク内のプログラム同士で短いデータを送り合う通信方式）で機体制御ソフトへ渡す。これにより、画像認識と機体制御を別々のプログラムとして開発・更新できる。



図1 開発環境と基本構成



図2 Vision Studio の処理の流れ

※ 本レポートでは処理の考え方と構成のみを示し、ソースコードおよび調整値は掲載しない。

2.2 Vision Studio 本体

項目	内容
正式名称	松戸技術革新センター カメラ認識技術検証アプリケーション（略称 Vision Studio）
版	v0.2.3（2026-09-21）
実行環境	Python（Thonny）。PC で開発、Raspberry Pi 4B で実行。追加の画像ライブラリに依存しない軽量構成
モード	① 色検知 ② 物体追跡 ③ ライントレース ④ アーム接近 ⑤ QR／バーコード読取
出力	JSONL ログ／UDP／シリアル
構成方針	機体制御ソフトとは別プロセス。UDP で状態を渡し、機体制御ソフトは改変しない
搭載先	CWM・CBV（ともに Raspberry Pi 4B、最新 OS）。CTR は対象外

2.3 アーム接近モードの考え方

ピッキングロボット CWM の水平多関節（SCARA 型）アームで物をつかむには、対象物がアームに対して「左右どちらに・どれくらい離れて」あるかが必要になる。アーム接近モードでは、アーム先端付近に取り付けたカメラの画像から対象物を色や形で検出し、画面の中心に対する対象物の位置のずれと、映って

いる大きさを求める。位置のずれはアームを旋回させる方向の目安に、大きさは対象までの近さの目安になる（近いほど大きく映る）。これらを UDP またはシリアルで出力し、アームの制御側で使う。



図3 アーム接近モードの処理の流れ

2.4 認識対象と手法の方針

- ・ 認識したいもの：自前で撮影・学習させる物体 5 種類程度、ライン、走行経路上の人、障害物、QR コード、バーコード
- ・ 物体学習は Teachable Machine → TFLite の流れで進める（YOLOv5 はラズパイ 4B では重いと判断）。学習型の物体検知は M-TIC Detect（MTIC-DR-2026-008）へ分離
- ・ 認識結果を反映する先は、ラズパイの GUI から Pico 3 台へ CAN で指令する構成（CAN は MTIC-TR-2026-001 参照）

3. 検証方法と結果

3.1 検証方法

PC の Thonny 上で USB カメラを接続し、5 モードの切替と各モードの認識表示、3 種の出力を確認した。

3.2 結果

項目	結果
5 モードの切替・表示	PC 上で動作確認（v0.2.3）
JSONL／UDP／シリアル出力	PC 上で出力を確認
アーム接近の位置ずれ・大きさの出力	PC 上で出力を確認。アームを実際に動かした検証は未実施

なお、各モードの認識精度、Raspberry Pi 4B 上の処理速度 [fps] は今回は計測していない。今後、5.1 節の検証計画に沿って計測し、そのデータをもとに評価する予定である。

4. 課題と考察

- ・ **実機での確認**：PC 上の確認にとどまる。ラズパイ 4B での処理速度、照明条件による色検知の変動が主要な不確定要素。
- ・ **学習型認識の分離**：Vision Studio 単体では自作学習を持たず、学習型は M-TIC Detect に分けた。両アプリの出力を機体側でどう統合するかは未設計。
- ・ **カメラ切替の重さ**：カメラ切替が重い問題に対し、共通カメラモジュールを他アプリで先行導入済みであり、本アプリへの適用は次版で行う。

5. 今後の展望

5.1 検証計画

計測項目	方法	判定基準
処理速度	ラズパイ 4B で各モードの fps を記録	制御に必要な fps を満たす
色検知の安定性	照明 3 条件（昼・夜・蛍光灯）で検出率を記録	各条件で検出率を記録し基準を設定
アーム接近	SCARA 型アームに搭載し、対象物への接近・把持を 10 回試行	成功回数／10、失敗の原因を記録
QR／バーコード	距離・角度を変えて読取可否を記録	読取可能距離・角度

5.2 応用の可能性

将来的には販売できる形を目指す。カメラ認識を GUI で確認しながら調整できる構成は、他の機体や用途での認識技術の検証にも使える可能性がある。

5.3 カメラ認識技術の統合（構想）

M-TIC ではカメラを使った認識技術を、床面認識（M-TIC AutoDrive）、ライン認識（CWM_CamLine）、物体検知（M-TIC Detect）、色・形の認識（Vision Studio）として、それぞれ単体で開発・検証してきた。今後はこれらを最後に組み合わせることで、自動運転の技術に使える可能性がある。たとえば、床面認識で走れる範囲を把握しながら物体検知で走行中の障害物を見つけて避ける、人がロボットの経路を横切ったときに物体検知で人と判定して減速・停止する、ラインのある区間ではライン認識、ない区間では床面認識に切り替えて走る、といった使い方が考えられる。まずは単体ごとの精度と処理速度を確かめたうえで、組み合わせ方を段階的に検討していきたい。

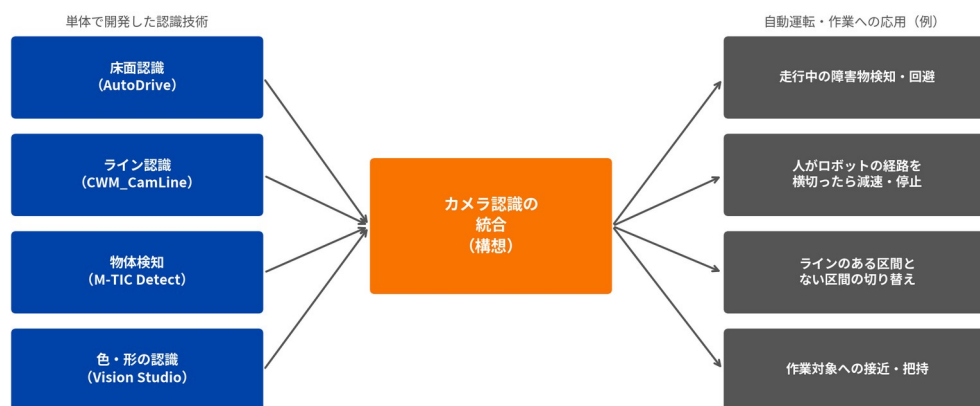


図4 カメラ認識技術の統合の構想（各レポートの技術を組み合わせた応用の例）

6. まとめ

5つの認識モードと3種の出力を1つのGUIに統合した Vision Studio v0.2.3 を開発し、PC 上で動作を確認した。アーム接近モードでは、対象物の位置のずれと大きさをアーム制御の目安として出力できる。認識精度・処理速度・アームへの搭載はいずれもまだ確かめておらず、次段階で CWM・CBV 上で計測したい。

参考資料

OpenCV ドキュメント (opencv.org)

Google, “Teachable Machine”／TensorFlow Lite ドキュメント

改訂履歴 初版 (2026-09-26)